

课程设计计划：掀起 AI 编程的“引擎盖”——从机制到复现

Context

项目背景： Claude Code 复现项目，分为两条线： 1. **工程复现**（他人负责）：复现 Claude Code 工具本身的架构和实现 2. **模型逻辑复现**（本课程目标）：理解并复现 Claude Code 背后模型（Opus 4.6）的编程能力

核心目标： 不是训练一个 Opus 4.6，而是基于开源模型（Qwen2.5-Coder-7B）复现其在编程任务上的核心能力。

技术路线： - **基础模型：** Qwen2.5-Coder-7B-Instruct（7.61B 参数，Dense 架构，Apache 2.0） - **目标基准：** SWE-bench Verified 40%+（在 7B 级别模型中达到 SOTA） - **复现方法：** Prompt Engineering + LoRA Fine-tune + Agent 架构设计 - **硬件需求：** 单卡 A800（80GB）即可完成 LoRA fine-tune

课程总纲

课程名称： 掀起 AI 编程的“引擎盖”——从机制拆解到模型复现

课程定位

- **受众：** 算法工程师、模型研究者、AI 系统架构师
- **前置要求：** 熟悉 LLM 基础、有 PyTorch/transformers 使用经验、了解 fine-tune 流程
- **核心目标：**
 1. 理解 Claude Code 背后的模型机制（怎么做到的）
 2. 掌握用开源模型复现这些能力的完整路径（怎么复现）
 3. 能独立构建一个在 7B 级别达到 SOTA 的编程代理

课程结构： 4 大篇章 + 12 个模块

第一篇：机制拆解 — Opus 4.6 是怎么做到的（模块 0-2）

模块 0：核心循环 — AI 编程代理的“发动机”

目标：理解 Claude Code 背后的完整工作机制

主题	内容
0.1 从一个真实任务说起	Trace 回放：Claude Code 修复一个 bug 的完整过程（Grounding → Planning → Action → Feedback）
0.2 核心循环推导	Intent → Grounding（感知现状）→ Planning（规划路径）→ Action（执行变更）→ Grounding（观察结果）→ Feedback（评估修正）
0.3 MDP 建模	将编程代理建模为马尔可夫决策过程：状态=代码库快照，动作=工具调用，奖励=测试结果
0.4 Claude Code 与 Opus 4.6 的交互协议	System Prompt、Tool Definitions、Tool Use Protocol、Message Structure、Streaming、Context Management、Permission、Hooks
0.5 Opus 4.6 的能力清单	长上下文推理、工具调用、代码理解、错误恢复 — 哪些是模型能力，哪些是工程设计

关键论文：ReAct (Yao 2022), Chain-of-Thought (Wei 2022)

实操：手动 trace 一个 Claude Code session，标注每一步属于循环的哪个环节

Claude Code ↔ Opus 4.6 交互协议详解：

协议层	内容	技术细节
1. System Prompt 结构	身份定义、工具说明、约束规则、响应风格	XML 标签分段、关键指令重复强调、CLAUDE.md 自动加载
	15+ 工具的 JSON	

协议层	内容	技术细节
2. Tool Definitions 协议	Schema 定义	Read/Write/Edit/Bash/Grep/Glob/Agent/WebSearch/MCP 等，描述详细度是性能关键
3. Tool Use 协议	tool_use → tool_result 循环	<pre>{"type": "tool_use", "id": "...", "name": "Read", "input": {...}} → {"type": "tool_result", "tool_use_id": "...", "content": "..."} </pre>
4. Message 结构	user/assistant/system 三角色	tool_result 必须在 content 数组首位，支持并行工具调用
5. Streaming 协议	Server-Sent Events (SSE)	message_start → content_block_delta (text/input_json) → message_stop
6. Context Management	200K 窗口，60% 开始降级	Auto-Compact（自动压缩）、Subagent 隔离、CLAUDE.md 持久化
7. Permission 协议	5 种权限模式 + 工具级限制	default/plan/acceptEdits/dontAsk/bypassPermissions, Bash 禁网络工具
8. Hooks 协议	18 种事件钩子	PreToolUse（可阻断）、PostToolUse、UserPromptSubmit、SessionStart 等，支持 Command/HTTP/Prompt/Agent 四种 handler
9. MCP 集成	Model Context Protocol 扩展	工具命名 <code>mcp__<server>__<action></code> ，settings.json 配置
10. Subagent 协议	隔离上下文的子代理	独立 transcript，主代理只收到摘要，可用 worktree 隔离文件系统

数据流：

User Input → [UserPromptSubmit Hook] → Build Message + System Prompt → Claude API (streaming) → Parse tool_use → [PreToolUse Hook]

→ Permission Check → Execute Tool → [PostToolUse Hook]
→ Stream tool_result → Model Response → [Stop Hook] → User

模块 1：Grounding — 感知的策略空间与实现

核心问题：模型怎么“看到”代码库？Opus 4.6 用了什么策略？

主题	内容
1.1 为什么 Grounding 是瓶颈	SWE-agent 的核心洞察：Agent-Computer Interface 比模型更重要
1.2 策略 1：暴力检索	Glob/Grep/全文件读取 — Claude Code 的方案，简单有效但消耗 token
1.3 策略 2：结构化感知	AST 解析、LSP 语义信息 — OpenCode 的方案，精准但实现复杂
1.4 策略 3：向量检索 (RAG)	Embedding + 向量数据库，适合超大代码库，但有语义漂移风险
1.5 Opus 4.6 的 Grounding 能力	200K 上下文窗口、代码结构理解、跨文件依赖追踪 — 这些能力怎么来的
1.6 复现路径	Qwen2.5-Coder-7B 的 Grounding 能力现状 + 如何通过 prompt/fine-tune 补齐

关键论文：SWE-agent (Yang 2024), Agentless (Xia 2024)

实操：- 对比 Claude Code 和 Qwen2.5-Coder-7B 在同一任务上的 grounding 行为 - 测量不同策略的 token 消耗和信息覆盖率

模块 2：模型能力拆解与复现路径

核心问题：Opus 4.6 的编程能力从哪来？Qwen2.5-Coder-7B 差在哪？怎么补？

主题	内容
2.1 代码预训练	从 Codex 到 Opus 4.6：在代码上预训练为什么有效？训练数据、目标函数、架构选择
2.2 指令遵循与 RLHF	模型怎么学会“听话”的 — RLHF、Constitutional AI、DPO 的对比

主题	内容
2.3 Tool Use 能力	函数调用的训练机制：从自然语言到结构化 JSON 输出，训练数据构造
2.4 长上下文推理	200K 窗口的技术实现（RoPE、ALiBi、Yarn）与实际表现衰减
2.5 Qwen2.5-Coder-7B 能力现状	7.61B Dense 架构、128K 上下文、在 5.5T tokens 代码数据上预训练、Apache 2.0 开源
2.6 能力 Gap 分析	Opus 4.6 vs Qwen2.5-Coder-7B：哪些能力已具备、哪些需要补、怎么补
2.7 三条复现路径	Prompt Engineering（快速验证）/ LoRA Fine-tune（单卡 A800）/ 全参数 Fine-tune（2-3 卡 A800）

关键论文： Codex (Chen 2021), RLHF (Ouyang 2022), Toolformer (Schick 2023)

实操： - 用 Qwen2.5-Coder-7B 跑 SWE-bench 样例，分析失败 case - 识别能力短板（规划、工具调用、错误恢复等）

第二篇：策略空间 — 每个环节的实现选择（模块 3-6）

模块 3：Planning — 规划的策略空间与复现

核心问题： 知道现状和目标后，怎么拆解路径？Opus 4.6 用什么策略？

主题	内容
3.1 扁平任务列表	最简方案：直接列出 TODO，线性执行 — ReAct 的核心
3.2 树状分解	Tree of Thoughts：展开多个方案分支，评估后选最优
3.3 DAG 依赖图	任务间有依赖关系时的拓扑排序执行
3.4 Opus 4.6 的 Planning 能力	从 trace 反推：它用的是哪种策略？什么时候细规划、什么时候粗规划？
3.5 Qwen2.5-Coder-7B 的 Planning 现状	能力评估：能否生成合理的任务分解？失败模式是什么？
3.6 复现路径	

主题	内容
	Prompt 设计：如何引导模型做好 Planning? Fine-tune 数据构造：Planning 轨迹的标注

关键论文： Tree of Thoughts (Yao 2023), MetaGPT (Hong 2023)

实操： - 对比 Opus 4.6 和 Qwen2.5-Coder-7B 在同一任务上的 Planning 质量 - 设计 Planning-focused prompt，测试改进效果

模块 4：Action — 执行的策略空间与复现

核心问题： 决定了做什么之后，怎么做？工具调用能力怎么训练？

主题	内容
4.1 策略 1: Tool Use (JSON 函数调用)	Claude Code 的核心方案：Read/Edit/Write/Bash 工具集
4.2 策略 2: CodeAct (代码执行)	用 Python/Shell 代码作为动作语言，比 JSON 更灵活
4.3 工具设计原则	什么样的工具接口对模型友好？从 SWE-agent 的 ACI 设计学到的
4.4 Opus 4.6 的 Tool Use 能力	工具调用的准确率、参数填充的正确性、错误恢复能力
4.5 Qwen2.5-Coder-7B 的 Tool Use 现状	原生支持 function calling 吗？调用准确率如何？
4.6 复现路径	Tool Use 训练数据构造：(任务, 工具调用序列) 对的生成；LoRA fine-tune 工具调用层

关键论文： CodeAct (Wang 2024), SWE-agent (Yang 2024), Toolformer (Schick 2023)

实操： - 构造 100 条工具调用训练样本 - 用 LoRA fine-tune Qwen2.5-Coder-7B，对比 fine-tune 前后的工具调用准确率

模块 5：Feedback — 评估与修正的策略空间与复现

核心问题： 怎么知道做对了？做错了怎么办？自我修正能力怎么训练？

主题	内容
5.1 策略 1：外部验证	测试执行、Lint 检查、类型检查 — 最可靠但覆盖面有限
5.2 策略 2：模型自评	LLM 审查自己的输出 — Self-Refine 的机制
5.3 策略 3：Reflexion	从失败中学习：将错误经验转化为语言记忆，下次避免
5.4 Opus 4.6 的自我修正能力	遇到测试失败时的恢复策略、错误诊断能力
5.5 Qwen2.5-Coder-7B 的 Feedback 现状	能否从测试失败中学习？能否识别自己的错误？
5.6 复现路径	Reflexion 训练数据构造：(失败案例, 反思, 修正) 三元组；强化学习 vs 监督学习

关键论文： Self-Refine (Madaan 2023), Reflexion (Shinn 2023)

实操： - 收集 Qwen2.5-Coder-7B 的失败案例，分析失败模式 - 设计 Reflexion prompt，测试自我修正能力提升

模块 6：搜索策略 — 从贪心到 MCTS 的复现路径

核心问题： 当前编程代理的搜索策略处于什么水平？如何实现更高级的搜索？

主题	内容
6.1 搜索策略进化光谱	贪心 (ReAct) → 线性回溯 (Reflexion) → 树搜索 (ToT) → MCTS
6.2 当前主流为什么是贪心	实际工程约束：token 成本、延迟、状态管理复杂度
6.3 代码空间搜索的特殊困难	无界状态空间、模糊评价函数、高分支因子
6.4 实现树搜索的工程方案	如何用 Qwen2.5-Coder-7B 实现 Tree of Thoughts？状态管理、分支评估、剪枝策略
6.5 MCTS 的简化实现	用测试通过率作为价值函数、用 LLM 作为策略网络，实现一个最小 MCTS 编程代理

关键论文： LATS (Zhou 2023), Tree of Thoughts (Yao 2023)

实操： - 实现一个基于 Tree of Thoughts 的编程代理 - 对比贪心 vs 树搜索在复杂任务上的成功率

第三篇：复现实战 — 从零到 70% SWE-bench（模块 7-9）

模块 7：Prompt Engineering 复现路径

核心问题：不改模型权重，纯靠 prompt 能复现多少能力？

主题	内容
7.1 System Prompt 设计	为 Qwen2.5-Coder-7B 设计编程代理的 system prompt：角色定义、工具说明、行为约束
7.2 ReAct Prompt 模板	Thought → Action → Observation 循环的 prompt 实现，few-shot examples 的选择
7.3 工具调用的 Prompt 设计	JSON schema 定义、参数填充引导、错误处理提示
7.4 上下文管理策略	有限窗口下的信息压缩：代码摘要、关键信息提取、优先级排序
7.5 常见失效模式与修正	API 幻觉、过度工程、循环死锁、token 浪费 — 如何通过 prompt 约束
7.6 Baseline 建立	用纯 prompt 的 Qwen2.5-Coder-7B 跑 SWE-bench，建立性能基线

实操： - 设计完整的 ReAct prompt 模板 - 在 10 个 SWE-bench 样例上测试，记录成功率和失败模式 - 迭代优化 prompt，提升成功率

模块 8：LoRA Fine-tune 复现路径

核心问题：如何高效地让 Qwen2.5-Coder-7B 学会编程代理的行为模式？

主题	内容
8.1 训练数据构造	

主题	内容
	从 SWE-bench 构造 (任务描述, 工具调用轨迹, 最终代码) 三元组; 数据清洗与质量控制
8.2 LoRA 配置选择	Rank、Alpha、Target Modules 的选择; 针对编程任务的最优配置
8.3 训练目标设计	Next-token prediction vs Trajectory-level reward; 损失函数的设计
8.4 硬件配置与训练	单卡 A800 的 LoRA fine-tune 实战: DeepSpeed、Gradient Checkpointing、混合精度训练
8.5 评估与迭代	在 SWE-bench Verified 上评估; 分析 fine-tune 后的能力提升和新增失败模式
8.6 目标: 在 7B 级别达到 SOTA	通过 LoRA fine-tune 最大化 Qwen2.5-Coder-7B 的编程代理能力

实操: - 构造 500-1000 条高质量训练样本 - 用单卡 A800 进行 LoRA fine-tune (约 1-2 天) - 对比 fine-tune 前后在 SWE-bench 上的表现

模块 9: Agent 架构设计与工程落地

核心问题: 如何把 fine-tuned 模型包装成一个可用的编程代理?

主题	内容
9.1 Agent 架构设计	核心循环实现: Grounding → Planning → Action → Feedback; 状态管理、工具注册、错误恢复
9.2 工具接口设计	参考 SWE-agent 的 ACI 设计: Read、Edit、Search、Test、Bash 工具的实现
9.3 上下文窗口管理	动态裁剪策略: 保留关键信息、压缩历史、优先级队列
9.4 错误恢复机制	工具调用失败、测试不通过、陷入循环 — 各种异常的处理策略
9.5 评估与对比	在 SWE-bench 完整集上评估; 与 Claude Code、OpenCode、Devin 对比

主题	内容
9.6 部署与优化	推理加速（vLLM、TensorRT-LLM）、成本控制、API 封装

实操（结业项目）： - 从零实现一个完整的编程代理（基于 fine-tuned Qwen2.5-Coder-7B） - 在真实项目上测试：修复 bug、添加功能、重构代码 - 撰写技术报告：复现路径、性能对比、经验总结

第四篇：工程落地与前沿展望（模块 10-11）

模块 10：真实世界工作流与工程落地

核心问题：如何在真实团队和项目中落地自研的编程代理？

主题	内容
10.1 独立开发者工作流	修复 Bug → 测试 → 提交代码 → 发起 PR，全流程由代理驱动
10.2 团队集成	CI/CD 钩子、让代理参与 Code Review、PR 自动化、与 GitHub/GitLab 的集成
10.3 Monorepo 与大型代码库策略	应对超大上下文、针对性的代码探索、分层配置
10.4 何时“不”该用代理	了解局限性、典型失败场景、知道什么时候必须手动写代码
10.5 安全、隐私与合规	数据流向（哪些上云、哪些留本地）、API 密钥管理、审计日志、代码泄露风险
10.6 成本控制	Token 用量监控、模型降级策略、缓存与去重、自托管 vs API 调用的成本对比

实操： - 为一个真实项目配置 CI/CD 集成的代理工作流 - 成本分析：对比自研代理 vs Claude Code API 的成本

模块 11：前沿方向与持续演进

核心问题：编程代理的下一步是什么？如何持续改进？

主题	内容
11.1 多模态编程代理	理解 UI 截图、设计稿、架构图 — 视觉输入如何改变编程代理
11.2 超长上下文 (1M+ tokens)	Gemini 1.5 Pro 的 1M 窗口对编程代理意味着什么？如何利用？
11.3 持续学习与个性化	从用户反馈中学习、适应团队代码风格、记忆项目特定知识
11.4 多代理协作	MetaGPT 式分工、AutoGen 式对话 — 复杂任务的多代理分解
11.5 从 SWE-bench 到真实世界	Benchmark 与实际使用的 gap；如何构建更真实的评估体系
11.6 搜索策略的未来	MCTS + 自我对弈 + 强化学习在编程代理中的应用前景

关键论文： Gemini 1.5 (Google 2024), MetaGPT (Hong 2023), LATS (Zhou 2023)

讨论： 学员分享各自的复现经验、遇到的问题、未来改进方向

附录：课程论文清单（层次关系）

多代理协作层 (Multi-Agent)
搜索策略层 (Search Strategy)
编程代理层 (Programming Agent)
反思层 (Reflection)
基石层 (Foundation)
模型底座层 (Model Foundation)

模型底座层 (Model Foundation)

论文	年份	核心贡献
Codex	2021	代码预训练

论文	年份	核心贡献
RLHF (InstructGPT)	2022	人类反馈对齐
Constitutional AI	2022	原则约束的安全机制

基石层 (Foundation)

论文	年份	核心贡献
Chain-of-Thought Prompting	2022	逐步推理，代理“思考”的基础
ReAct	2022	推理+行动交织，Agent 循环的起源
Toolformer	2023	LLM 自主使用工具的理论基础

反思层 (Reflection)

论文	年份	核心贡献
Self-Refine	2023	生成→自评→改进的迭代循环
Reflexion	2023	语言记忆式经验学习
Tree of Thoughts	2023	推理从链式扩展为树状搜索

编程代理层 (Programming Agent)

论文	年份	核心贡献
SWE-agent	2024	ACI 设计比模型更重要
CodeAct	2024	代码执行作为行动方式
Agentless	2024	简单两步流程的有效性

搜索策略层 (Search Strategy)

论文	年份	核心贡献
LATS	2023	LLM 同时充当策略网络和价值网络
Tree of Thoughts	2023	树状搜索在推理任务中的应用

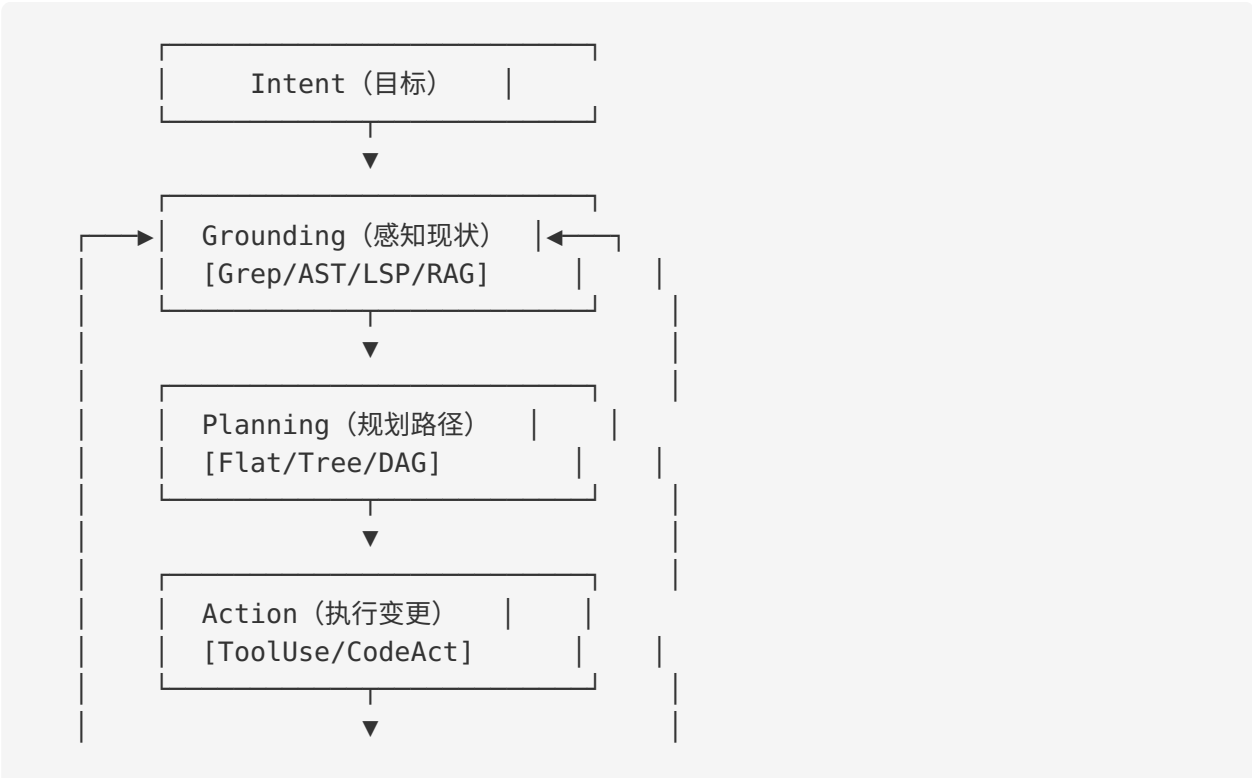
多代理协作层 (Multi-Agent)

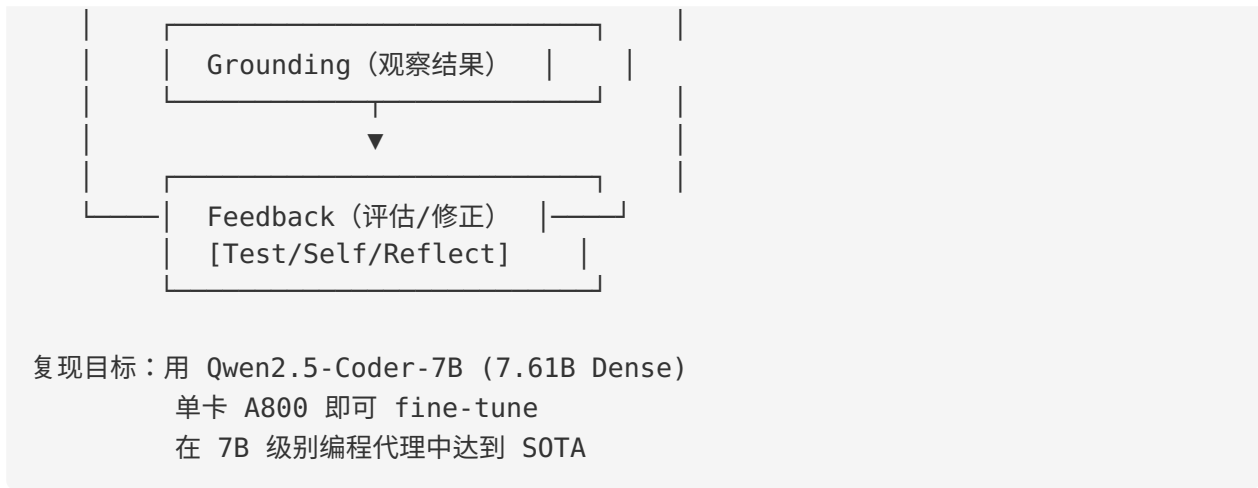
论文	年份	核心贡献
MetaGPT	2023	角色分工式多代理协作
AutoGen	2023	对话式多代理框架

课程形式建议

- 4 大篇章，11 个模块（模块 0-11）
- 每模块 2.5-3 小时（共约 28-33 小时）
- 每模块结构：
 - 机制讲解（25%）：这个环节是什么、为什么重要
 - Opus 4.6 能力分析（20%）：它是怎么做到的
 - Qwen2.5-Coder-7B 现状（15%）：差距在哪
 - 复现路径（20%）：怎么补齐差距
 - 动手实操（20%）：实际编码和实验
- 结业项目：从零复现一个在 7B 级别达到 SOTA 的编程代理

核心一张图（课程开篇使用）





课程特色

1. **双线并进**：机制拆解 + 复现实战，理论与实践紧密结合
2. **以终为始**：从 Opus 4.6 的能力反推实现路径，目标明确
3. **可落地**：基于 Qwen2.5-Coder-7B + 单卡 A800，成本极低，学员可独立复现
4. **完整闭环**：从 prompt 到 fine-tune 到 agent 架构，覆盖全流程
5. **前沿视角**：不只是复现现有能力，还探讨未来方向（MCTS、多模态、超长上下文）